Industrie- und Handelskammer
für Ostfriesland und Papenburg

Bestätigung über die durchgeführte Projektarbeit im IT- Ausbildungsberuf

Fachinformatiker für Anwendungsentwicklung

Abschlussprüfung: Sommer 2026 Winter

Azubi-Identnummer: 121/ 572525

Ausbildungsfirma:

Name : Hamburg

Vorname: Niklas

Straße: Kapitän-Rüter-Str. 81

PLZ / Ort: 26871 Papenburg

Firma : journaway GmbH

Straße : Am alten Handelshafen 1

PLZ / Ort: 26789 Leer

Projektbezeichnung: Bereitstellung eines Microservices zur Synchronisation von dezentralisiert gespeicherten Kundendaten

Projektbeginn: 29.03.2026 Projektfertigstellung: 05.05.2026 Zeitaufwand in Std. 80

Bestätigung der Ausbildungsfirma:

Wir bestätigen, dass der Prüfling das oben bezeichnete Projekt einschließlich der Dokumentation im oben genannten Zeitraum selbständig ausgeführt hat.

Leer, 05.05.2026 — TStamma

Ort, Datum — Unterschrift Ausbildender

Alicante, 05.05.2026

Ort, Datum — Unterschrift Projektverantwortliche/r

Eidesstattliche Erklärung des Prüflings:

Ich versichere, dass ich die Fachaufgabe selbständig durchgeführt und den dazugehörenden Report selbständig erstellt habe.

Papenburg, 05.05.2026 — N.Hamburg

Ort, Datum — Unterschrift Prüfling

Diese Bestätigung ist als 1. Seite der Dokumentation hochzuladen!

Abschlussprüfung Sommer 2026

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit

Bereitstellung eines Microservices

zur Synchronisation von dezentralisiert gespeicherten Kundendaten

Abgabetermin: Papenburg, den 08.05.2026

Prüfungsbewerber:
Niklas Hamburg
Kapitän-Rüter-Straße 81
26871 Papenburg

Ausbildungsbetrieb:
JOURNAWAY journaway GmbH
Am alten Handelshafen 1
26789 Leer

Inhaltsverzeichnis

Abbildungsverzeichnis **III**

Tabellenverzeichnis **IV**

Abkürzungsverzeichnis **V**

1 Einleitung **1**
1.1 Projektumfeld . . . 1
1.2 Projektziel . . . 1
1.3 Projektbegründung . . . 1
1.4 Projektschnittstellen . . . 2
1.5 Projektabgrenzung . . . 2

2 Projektplanung **2**
2.1 Projektphasen . . . 3
2.2 Abweichungen vom Projektantrag . . . 3
2.3 Ressourcenplanung . . . 3
2.4 Entwicklungsprozess . . . 4

3 Analyse **4**
3.1 Ist-Analyse . . . 4
3.2 Wirtschaftlichkeitsanalyse . . . 5
3.3 Anforderungsanalyse . . . 6
3.4 Nutzwertanalyse . . . 7

4 Entwurf **7**
4.1 UML-Aktivitätsdiagramm . . . 8
4.2 Architekturdesign . . . 8
4.3 Datenbankstruktur . . . 9

5 Implementierung **9**
5.1 Implementierung der Datenstrukturen . . . 10
5.2 Implementierung RabbitMQ . . . 10
5.3 Implementierung des HTTP-Clients . . . 10
5.4 Implementierung der Synchronisationshistorie . . . 11
5.5 Implementierung der Geschäftslogik . . . 11

6 Abnahmephase **12**
6.1 Unittests . . . 12
6.2 Integrationstests . . . 12
6.3 Test Ergebnisse . . . 13

7 Deployment **13**

8 Dokumentation **13**
8.1 Entwicklerdokumentation 14

9 Fazit **14**
9.1 Soll-/Ist-Vergleich 14
9.2 Lessons Learned 15
9.3 Ausblick 15

Literaturverzeichnis **16**

Eidesstattliche Erklärung **17**

A Anhang **i**
A.1 Detaillierte Zeitplanung i
A.2 Synchronisations Abläufe ii
A.3 Datenbankmodell iii
A.4 RabbitMQ iv
A.5 HTTP-Client viii
A.6 Synchronisationshistorie x
A.7 Geschäftslogik xiii
A.8 Tests xvii
A.9 Pipelines und Automatisierung xx
A.10 Entwicklerdokumentation xxiv

Abbildungsverzeichnis

1 Nightcrawler ER-Diagramm . . . iii
2 RabbitMQ DI Registrierung . . . iv
3 Queue Initialisierung Hauptmethode . . . v
4 Event Publisher Hauptmethode . . . vi
5 Event Consumer Hauptmethode . . . vii
6 HubSpotApiClient . . . viii
7 HubSpotApiRateLimiter . . . ix
8 DbOperation SyncHistory . . . x
9 Repository SyncHistory . . . xi
10 Service SyncHistory . . . xii
11 PersonalInformation Upsert Logik . . . xiii
12 API Logik . . . xiv
13 RegEx Aufruf . . . xv
14 CommunicationPreference Logik . . . xvi
15 Nightcrawler Projekt Struktur . . . xvii
16 Testabdeckung . . . xvii
17 Unit Test . . . xviii
18 Integrationstest . . . xix
19 Tests und statische Codeanalyse . . . xx
20 Docker Images vorbereiten . . . xxi
21 Versionierung und Deployaufruf . . . xxii
22 Deployment mit SSH . . . xxiii
23 Ausschnitt Entwicklerdokumentation . . . xxiv

Tabellenverzeichnis

1 Zeitplanung 3
2 Vorher-/Nachher-Vergleich 7
3 Soll-/Ist-Vergleich 15

Abkürzungsverzeichnis

API	Application Programming Interface
CRM	Customer Relationship Management
DLQ	Dead-Letter-Queue
TTL	Time-To-Live
ERD	Entity-Relationship-Diagramm
EF-Core	Entity-Framework-Core
DI	Dependency Injection
URL	Uniform Resource Locator
RegEx	Regular Expression
Json	JavaScript Object Notation
SSH	Secure Shell

1 Einleitung

In dieser Dokumentation wird das Projekt 'Bereitstellung eines Microservices zur Synchronisation von dezentralisiert gespeicherten Kundendaten' beschrieben, welches im Rahmen der Prüfung zum Abschluss der Ausbildung zum Fachinformatiker Anwendungsentwicklung durchgeführt wurde. Es wird der Prozess der Entwicklung einer Anwendung dargestellt, die als eigenständiger Microservice implementiert wird.

1.1 Projektumfeld

JournawayGmbH ist ein in Leer ansässiges, 2017 gegründetes Reiseunternehmen und bietet Reisekomplett-Pakete auf ihrer Webseite an. Derzeit werden rund 160 Mitarbeiter beschäftigt und unter diesen Mitarbeitern arbeiten über ein Drittel in der internen IT-Abteilung, welche B2C und B2B Lösungen für das Unternehmen zur Verfügung stellt. Aufgaben dieser Abteilung liegen unteranderem darin die Webseite und die dazu gehörigen Daten, wie Buchungsdaten, Kundendaten etc. zu pflegen und zu ermöglichen, dass Preise und Informationen zu Leistungen innerhalb der Produkte stets aktuell sind.

Der Auftraggeber des Projektes ist die journaway GmbH selbst.

1.2 Projektziel

Journaway stellte in Auftrag ein Produkt zu entwickeln, welches als Schnittstelle Daten von zwei Kundendaten speichernden Systemen miteinander synchronisiert.

Mit der Entwicklung eines neuen Kundenportals auf der Webseite ist es notwendig, dass Informationen für die Kommunikation zwischen Unternehmen und Kunden nicht asynchron vorliegen. Teil dieser Aufgabe ist es einen Microservice zu entwickeln, der durch eventbasierte Synchronisierung die dezentrale Verfügbarkeit von Daten auflöst und eine einzige Source of Truth schafft, wodurch die Kundendaten auf beiden Systemen identisch sind.

1.3 Projektbegründung

In beiden bisherigen Systemen werden Kundendaten gespeichert, welche keine Relation zu einander haben und können somit für den gleichen Kunden unterschiedlich vorliegen. Mit dem neuen Microservice *Nightcrawler* sollen Fehler beim persistieren kundenkontaktrelevanter Informationen minimiert werden. Außerdem soll es ermöglichen das im bisherigen CRM *HubSpot* gespeicherte Kundendaten mit dem angemeldeten Kunden vom *user-service* verknüpft werden, wodurch im Kundenportal weitere, für den Kunden interessante Informationen dargestellt werden können.

Ohne den *Nightcrawler* müssten Mitarbeiter aus dem Kundensupport eigenhändig nach Abweichungen in Kontaktinformationen suchen und diese manuell einpflegen. Dies würde einen erhöhten Ressourcenaufwand und eine höhere Anfälligkeit für Fehler bedeuten. Ziel ist es, den Erhalt von zentralen und aktuellen Kontaktdaten, ohne manuellen Aufwand zu gewährleisten.

1.4 Projektschnittstellen

Die Anwendung interagiert über Event Queues durch RabbitMQ mit einem anderen Microservice *user-service*. Durch diesen Service ist es möglich, dass persönliche Informationen, Kommunikationspräferenzen oder Reisende von Websitebesuchern erstellt, bearbeitet und gelöscht werden können. Außerdem können die Nutzer dort auch ihre Buchungen einsehen.

Zudem kommuniziert die Anwendung über einen HttpClient mit dem CRM *HubSpot*. Dies ist der Ort an dem standardmäßig kundenbezogene Daten gespeichert werden. Dieser cloud-basierte Service von dem Unternehmen *HubSpot, Inc.* wird hauptsächlich für die Speicherung von Kontaktdaten, Deals und Ausführung von Workflows genutzt.

Dadurch, dass es sich um einen autonomen Synchronisationsservice handelt gibt es keine direkten Benutzer, jedoch werden events zur Synchronisierung entweder durch den Websitenutzer oder Mitarbeiter in *HubSpot* ausgelöst. Schließlich werden die Synchronisationshistorien zu Kunden in einer PostgreSql Datenbank persistiert.

Der *user-service*, als auch der *Nightcrawler* sind Microservices aufbauend auf dem .NET 10 Framework von Microsoft und werden auf Servern mit Linux basiertem Betriebssystem (Debian 12.13) ausgeführt. Der Quellcode ist in der Sprache CSharp geschrieben. Als Entwicklungsumgebung wird JetBrains Rider 2026 verwendet.

1.5 Projektabgrenzung

Teil dieses Projektes ist die Planung und Entwicklung eines funktionellen und testbaren Prototyps. Die Aufnahme der Anwendung in den Livebetrieb wird nicht behandelt.

2 Projektplanung

Um einen möglichst reibungslosen Ablauf in der Entwicklung des Microservices zu gewährleisten, braucht es eine gute Planung. In diesem Abschnitt wird daher aufgezeigt, welche Ressourcen (Zeit, Hardware) wie eingesetzt werden.

2.1 Projektphasen

Die Umsetzung des Projektes findet während zwei vier Wochen Sprints statt und wird gemäß der Vorgaben der IHK, wie in Tabelle 1 dargestellt, innerhalb von 80 Stunden bearbeitet:

Zeitplanung Tabelle 1 zeigt die grobe Zeitplanung.

Projektphase	Geplante Zeit
Analyse	8 h
Entwurf	6 h
Implementierung	49 h
Einführung	7 h
Dokumentation	10 h
Gesamt	**80 h**

Tabelle 1: Zeitplanung

Eine detailliertere Zeitplanung findet sich im Anhang A.1: Detaillierte Zeitplanung auf Seite i.

2.2 Abweichungen vom Projektantrag

Bei dem Inhalt des Projektes wird die Anbindung von Hangfire mit der Anbindung von RabbitMQ ausgetauscht, da bei der Planung erkannt wurde, dass Hangfire in der ersten Iteration nicht benötigt wird und sich bei der Umsetzung der Event Queues auf den Message Broker RabbitMQ geeinigt wurde.

2.3 Ressourcenplanung

Hardware

1. Büroarbeitsplatz
2. Netzwerk mit Internetzugang
3. Laptop und zwei weitere Bildschirme
4. Server für die Anwendung und Datenbank

Software

1. Betriebssystem Laptop: Windows 11
2. Entwicklungsumgebung: JetBrains Rider 2026
3. Message Broker Software: RabbitMQ

4. Datenbanksoftware: PostgreSQL 16
5. Zielbetriebssystem: Linux Debian 12.13
6. Software zum generieren und einsehen von Testdaten auf der Datenbank: HeidiSQL pgAdmin 4

Personal

1. Für die Qualitätssicherung während der Review- und Testphase werden zwei weitere Entwickler benötigt.

2.4 Entwicklungsprozess

Während der Umsetzung des Projektes wird ein agiler Prozess bzw. Scrum verfolgt. Dies ist ein Prozess, bei dem iterativ kleinere Abschnitte eines Auftrages fertiggestellt werden, wodurch schnell auf unvorhergesehene Probleme oder neue Anforderungen eingegangen werden kann. Scrum zeichnet sich durch eine hohe Flexibilität aus und eignet sich gut für kleine Projekte bzw. Teams.

3 Analyse

Im Folgenden wird zunächst die bisherige Situation im Unternehmen betrachtet. Darauf aufbauend wird untersucht, welchen wirtschaftlichen Nutzen das Projekt bietet. Anschließend werden die fachlichen und technischen Anforderungen an die Anwendung beschrieben. Zum Abschluss folgt eine Nutzwertanalyse, um den nicht-monetären Nutzen des Projektes zu bewerten.

3.1 Ist-Analyse

Aktuell werden kundenbezogene Daten in zwei voneinander getrennten Systemen gespeichert. Zum einen im *user-service*, welcher Daten für das Kundenportal verwaltet, und zum anderen im CRM-System *HubSpot*. Zwischen beiden Systemen besteht bislang keine eigenständig verwaltete und durchgängig nachvollziehbare Synchronisation. Dadurch kann es passieren, dass Kontaktinformationen eines Kunden in beiden Systemen unterschiedlich vorliegen.

Besonders problematisch ist dies bei Informationen, die für die Kommunikation mit dem Kunden relevant sind. Dazu zählen unter anderem die E-Mail-Adresse, der Vorname, der Nachname, die Anrede sowie Kommunikationspräferenzen. Werden diese Daten in einem der beiden Systeme geändert, ist nicht sichergestellt, dass die Änderung zuverlässig im jeweils anderen System übernommen wird.

Zusätzlich existiert bislang keine zentrale Stelle, die nachvollziehbar speichert, wann eine Synchronisation durchgeführt wurde, aus welchem System eine Änderung stammt und ob diese erfolgreich

verarbeitet wurde. Fehler bei der Übertragung könnten dadurch nur mit erhöhtem manuellem Aufwand erkannt und korrigiert werden. Ebenso gibt es bisher keine eigenständige technische Lösung, um mit temporären Fehlern, Systemausfällen oder Rate Limits der HubSpot API umzugehen.

Aus diesem Grund soll mit dem Projekt *Nightcrawler* ein eigenständiger Microservice entwickelt werden, der die Synchronisation zwischen beiden Systemen übernimmt. Hierzu soll eine PostgreSQL-Datenbank eingeführt werden, in der sowohl der aktuelle Synchronisationszustand zwischen Benutzer und Kontakt, als auch die Historie aller Synchronisationsereignisse gespeichert wird.

Außerdem soll die Kommunikation zwischen den beteiligten Systemen asynchron über RabbitMQ erfolgen. Dafür werden acht Event Queues benötigt: eine Queue für Änderungen vom *user-service* in Richtung *HubSpot*, eine Queue für Änderungen von *HubSpot* in Richtung *user-service* sowie sechs weitere Queues, die jeweils pro Richtung für zwei erneute Versuche und einen Fehlschlag zuständig sind.

Die bisherige Situation soll also dahingehend verbessert werden, dass Kundendaten zwischen beiden Systemen automatisch, nachvollziehbar und fehlertolerant synchronisiert werden können. Zusätzlich soll die neue Anwendung in der Lage sein, Konflikte bei gleichzeitigen Änderungen zu erkennen und nach festgelegten Regeln aufzulösen.

3.2 Wirtschaftlichkeitsanalyse

Die Entwicklung des Microservices verursacht zunächst einmalige Projektkosten in Form von Arbeitszeit. Für die Umsetzung des Projektes sind insgesamt 80 Stunden vorgesehen. Bei einem internen Verrechnungssatz von 35 Euro Netto pro Stunde ergeben sich folgende Entwicklungskosten:

$$80 \text{ Stunden} \times 35 \text{ Euro/Stunde} = 2800 \text{ Euro}$$

Dem gegenüber steht der Nutzen, der sich aus der automatisierten Synchronisation von Kundendaten ergibt. Ohne eine solche Lösung müssten Abweichungen zwischen den Datenbeständen manuell erkannt, überprüft und korrigiert werden. Dies würde insbesondere im Kundensupport und in der technischen Betreuung zusätzlichen Aufwand verursachen.

Zusätzlich sinkt durch die Anwendung das Risiko, dass veraltete oder fehlerhafte Kontaktdaten in einem der Systeme verwendet werden. Dadurch können Fehler in Folgeprozessen reduziert werden, beispielsweise bei der Kundenkommunikation oder bei der Darstellung von Informationen im Kundenportal.

Wird angenommen, dass durch die automatisierte Synchronisation wöchentlich 2,5 Stunden manueller Aufwand eingespart werden können, ergibt sich bei einem internen Stundensatz von 15,50 Euro Netto pro Stunde ein jährliches Einsparpotenzial von:

$$2{,}5 \text{ Stunden/Woche} \times 15{,}50 \text{ Euro/Stunde} \times 52 \text{ Wochen} = 2015 \text{ Euro/Jahr}$$

Neben dieser direkten Einsparung entsteht zusätzlicher Nutzen durch eine höhere Datenqualität, bessere Nachvollziehbarkeit und eine geringere Fehleranfälligkeit. Das Projekt ist damit wirtschaftlich sinnvoll, da die Entwicklungskosten mittelfristig durch eingesparte Arbeitszeit und verbesserte Prozesse ausgeglichen werden können.

3.3 Anforderungsanalyse

Damit das Projektziel erreicht werden kann, muss die Anwendung sowohl fachliche als auch technische Anforderungen erfüllen.

Fachlich muss der *Nightcrawler* in der Lage sein, Änderungen an Kundendaten zwischen dem *user-service* und *HubSpot* in beide Richtungen zu synchronisieren. Dabei sollen mindestens die Datenfelder *email*, *givenName*, *surname*, *salutation* und *CommunicationPreference* berücksichtigt werden.

Wenn ein Benutzer im *user-service* erstellt oder geändert wird, soll ein entsprechendes Ereignis verarbeitet und die dazugehörigen Daten in *HubSpot* aktualisiert werden. Wird ein Benutzer neu erstellt, soll zunächst geprüft werden, ob anhand der E-Mail-Adresse bereits ein passender Kontakt in *HubSpot* existiert. Ist dies der Fall, soll dieser Kontakt aktualisiert werden. Falls kein Kontakt gefunden wird, soll ein neuer Kontakt erstellt werden.

Wenn dagegen in *HubSpot* ein Kontakt geändert wird, soll diese Änderung ebenfalls verarbeitet und an den *user-service* weitergegeben werden. Kontakte aus *HubSpot*, für die kein passender Benutzer im *user-service* existiert, sollen zunächst ignoriert werden.

Damit jede Synchronisation nachvollziehbar bleibt, muss jedes verarbeitete Ereignis zunächst in einer Historie gespeichert werden. Diese *SyncHistory* soll Informationen zur Quelle des Ereignisses, zum Ereignistyp, zu Benutzer- und Kontaktzuordnung, zum Payload, zum Zeitpunkt des Auftretens, zum aktuellen Status sowie zur Anzahl der bisherigen Verarbeitungsversuche enthalten.

Zusätzlich muss ein *UserContactState* gespeichert werden. Dieser dient dazu, die Zuordnung zwischen Benutzer und HubSpot-Kontakt sowie den letzten bekannten Synchronisationsstand zu verwalten. Dabei sollen unter anderem die Zeitpunkte der letzten Änderung im *user-service*, der letzten Änderung in *HubSpot* und der letzten erfolgreichen Synchronisation gespeichert werden.

Technisch muss die Anwendung acht Event Queues über RabbitMQ verwenden. Da keine Nachrichten bei einem Neustart oder Ausfall verloren gehen sollen, müssen die Hauptqueues als durable Quorum Queues umgesetzt werden. Für temporäre Fehler soll eine Retry-Logik mit TTL-basierten Retry Queues eingesetzt werden. Nicht behandelbare Fehler sollen in einer Dead-Letter-Queue abgelegt werden.

Zur Konfliktbehandlung soll eine optimistische Strategie verwendet werden. Wenn Daten in beiden Systemen unterschiedlich geändert wurden, soll der Datenstand des *user-service* bevorzugt werden. Wird also eine Änderung aus *HubSpot* erkannt, die im Konflikt mit aktuelleren Daten aus dem *user-service* steht, soll diese Änderung nicht übernommen, sondern stattdessen eine neue Synchronisation in Richtung *HubSpot* angestoßen werden.

3.4 Nutzwertanalyse

Neben dem wirtschaftlichen Nutzen bietet das Projekt auch einen deutlichen nicht-monetären Mehrwert. Insbesondere die Datenqualität, die Nachvollziehbarkeit von Änderungen und die technische Stabilität der Synchronisation werden verbessert.

Vor der Umsetzung des Projektes besteht das Problem, dass Änderungen an Kundendaten nicht zentral nachvollzogen werden können. Außerdem ist die Synchronisation zwischen beiden Systemen nicht eigenständig abgesichert, wodurch Fehler bei der Übertragung schwerer zu erkennen sind. Mit dem *Nightcrawler* wird eine Lösung geschaffen, die Änderungen dokumentiert, Synchronisationszustände speichert und Fehlerfälle gezielt behandeln kann.

Zur Bewertung des Nutzwertes werden die Kriterien Datenkonsistenz, Nachvollziehbarkeit, Ausfallsicherheit, Wartbarkeit und Automatisierungsgrad betrachtet. Die folgende Matrix stellt einen Vorher-/Nachher-Vergleich dar:

Kriterium	Gewichtung	Vorher	Nachher
Datenkonsistenz	5	2	5
Nachvollziehbarkeit	3	1	5
Ausfallsicherheit	3	2	4
Wartbarkeit	2	2	4
Automatisierungsgrad	2	2	4
Gesamt:	**15**	**27**	**68**
Nutzwert:		**1,8**	**4,53**

Tabelle 2: Vorher-/Nachher-Vergleich

Aus den Bewertungen ergibt sich, dass der Nutzen der neuen Anwendung vor allem in der zuverlässigeren Verarbeitung, der besseren Transparenz und der geringeren manuellen Abhängigkeit liegt.

Das Projekt verbessert somit nicht nur die technische Grundlage für die Synchronisation von Kundendaten, sondern schafft auch eine stabilere und besser wartbare Lösung für zukünftige Erweiterungen.

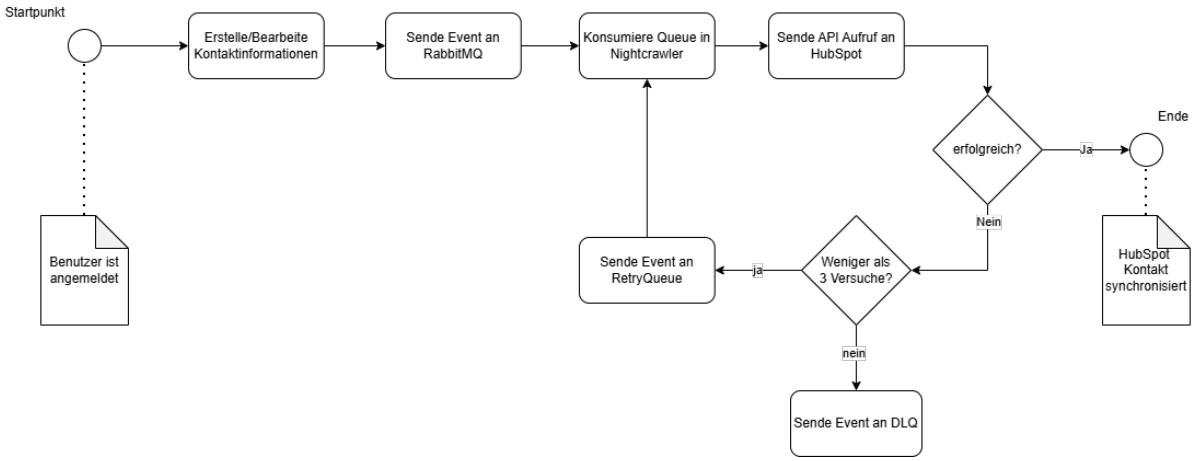
4 Entwurf

In dieser Phase soll die Implementierung vorbereitet werden. Dazu wird in einem UML-Aktivitätsdiagramm dargestellt, wie der Synchronisationsablauf in eine Richtung aussieht. Anschließend zeigt eine Archi-

tekturübersicht, wie diese Systeme miteinander integagieren. Am Ende stellt ein ER-Modell dar, wie die Datenbankstruktur für die Event Persistierung aussieht.

4.1 UML-Aktivitätsdiagramm

Das folgende UML-Aktivitätsdiagramm für die Synchronisation in Richtung *HubSpot*, zeigt den Ablauf der Synchronisation vom *user-service* ausgehend. Angefangen mit der Eingabe der Kontaktdaten vom Nutzer wird ein Event in die Main Queue geschickt. Anschließend konsumiert der *Nightcrawler* diese Queue und sendet basierend auf den Daten innerhalb der Event Message, einen API Upsert Aufruf an *HubSpot*. Sofern der API Aufruf einen 201 Created oder 200 Ok Statuscode zurückliefert wird das Event als erfolgreich angesehen. Andernfalls wird das Event in eine Retry Queue geschickt und nach einer 30 sekündigen TTL beim ersten Mal und 60 Sekunden beim zweiten Mal erneut ausgeführt. Wenn das zweite Mal ebenfalls nicht erfolgreich ist, wird das Event in eine DLQ geschickt.



Für genauere technische Abläufe siehe Anhang A.2: Synchronisations Abläufe auf Seite ii

4.2 Architekturdesign

Bei der ausgwählten Architektur handelt es sich um eine eine Orchestrator-Architektur, die es ermöglicht die Steuerung komplexer Arbeitsabläufe in Microservices zentralisiert auszuführen. Es gibt also ein Orchestrator Project, welches das Wissen über Konfigurationen beinhaltet, um andere Services zu registrieren und aufzurufen. Durch die Zentralisierung des Kontrollablaufs und die Entscheidungsgewalt über die nächsten Programmaufrufe, erleichtert der Orchestrator die Koordination verschiedener Services.

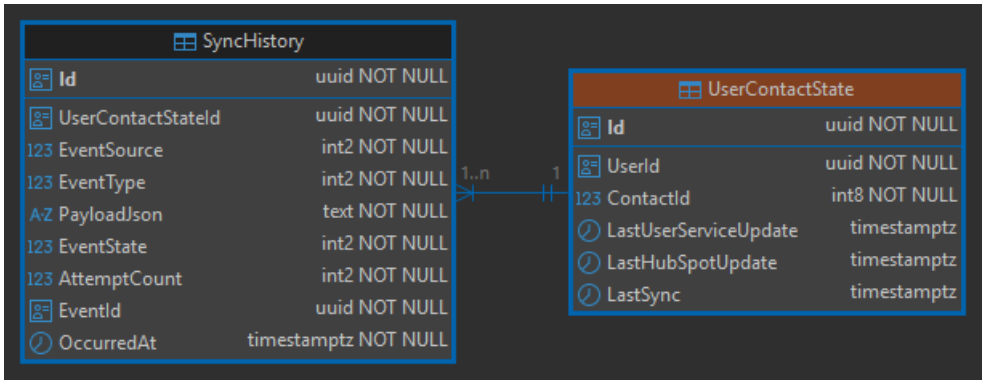
Die Entwicklung der Anwendung findet innerhalb des Microsoft .NET 10 Frameworks statt. .NET 10, veröffentlicht am 11. November 2025, ist ein plattformunabhängiges, modulares Framework, das die Entwicklung für Desktop, Web, Mobile, Cloud und IoT vereinfacht. Es basiert auf dem SDK-Modell,

das eine gleichzeitige Entwicklung und Ausführung auf Windows, Linux und macOS ermöglicht, und integriert die Laufzeitkomponenten CoreCLR, CoreFX sowie ASP.NET Core. Die Funktionsweise wird durch CSharp 14 geprägt, das Sprachfeatures wie Primary Constructors für alle Typen, Params-Spans für die Speicheroptimierung und Lambda-Verbesserungen bietet. Ein zentraler Meilenstein ist die Dateibasierte App-Unterstützung, die es erlaubt, einzelne *.cs-Dateien direkt auszuführen (dotnet run main.cs), ohne eine traditionelle Projektdatei zu benötigen. Dieses Framework wird regelmäßig erneuert, um den heutigen Standards gerecht zu werden. Außerdem bietet .NET eine große Auswahl an Funktionen und ermöglicht mithilfe von weiteren NuGet Paketen, wie zum Beispiel Entity Framework Core, eine vereinfachte Arbeit mit Entitäten aus Listen, Datenbanken und mehr.

4.3 Datenbankstruktur

Die Datenstruktur liegt zur Veranschaulichung im Folgenden als ERD vor. Sie besteht aus zwei Tabellen *SyncHistory* und *UserContactState*, welche in einer 1:n Beziehung zu einander stehen. Der *UserContactState* beschreibt einen Nutzer, welcher mit *HubSpot* verknüpft wird und die *SyncHistory* beschreibt alle Aktivitäten, die dieser Nutzer im Bezug auf dessen Daten gemacht hat.

Hier wird das Entity-Relationship-Diagramm (ERD) zur *nightcrawler* Datenbank dargestellt, welches Attribute, Relationen und die dazugehörigen Multiplizitäten enthält:



Anhang A.3: Datenbankmodell auf Seite iii

5 Implementierung

Dieser Abschnitt beschreibt, wie bei der Erstellung der Anwendung vorgegangen wurde. Zuerst wurden die Datenbanksturkturen und Anbindung implementiert. Danach wurde der Microservice erstellt. Anschließend wurden die Message Broker Abhängigkeiten und der HTTP-Client für die Verbindung zum CRM eingebaut. Schließlich wurden die Datenbankanfragen bzw. Synchronisationshistorie und die Geschäftslogik entwickelt.

5.1 Implementierung der Datenstrukturen

Die PostgreSQL Datenbank *nightcrawler* wurde händisch über *pgAdmin 4* angelegt und nutzt die *libpq-15.dll* Bibliothek. Im Anschluss wurden mithilfe des Paketes *EF-Core*, innerhalb der Applikation Migrationen aus Beziehungen zwischen Entitäten erstellt und auf die Datenbank angewandt. Ebenfalls wurden dadurch die Tabellen und Eigenschaften angelegt.

5.2 Implementierung RabbitMQ

Für die Verbindung zwischen dem Microservice *Nightcrawler* und dem Message Broker *RabbitMQ* wurde mithilfe von Serviceregistrierung in der DI-Klasse eine *IConnectionFactory* als Singleton mit den Applikationseinstellungen registriert. Durch die Registrierung als Singleton können alle Klassen, die eine Verbindung zu *RabbitMQ* benötigen, mit Dependency Injection dieselbe Instanz erzeugen. Dadurch müssen nicht alle Klassen die Einstellungen kennen, um die richtige *RabbitMQ* Instanz anzusprechen.

Um die verschiedenen Queues zu initialisieren wurde ein QueueInitializer erstellt, welcher als *IHosted-Service* implementiert wird, um beim Start der Applikation die Queues zu erstellen bzw. zu versichern, dass sie vorhanden sind. Innerhalb dieses Services wird die Topologie für *RabbitMQ* verwaltet. Hier werden die *MainQueues*, *RetryQueues* und *Dead-Letter-Queue (DLQ)'s* erstellt, die später für den Eventverkehr und das Errorhandling benötigt werden.

Als letztes werden EventPublisher und EventConsumer benötigt, um Events senden und empfangen zu können. Hierzu wurde im *user-service* auf Infrastruktur-Ebene ein EventPublisher implementiert, welcher indirekt nach einem API-Aufruf, die zu synchronisierenden Daten als Event sendet. Der *Nightcrawler* beinhaltet auf Presentations-Ebene den EventConsumer und empfängt die gesendeten Daten, um diese dann zu verarbeiten und mit *HubSpot* zu synchronisieren.

Screenshots der Anwendung in Bezug auf *RabbitMQ* befinden sich im Anhang A.4: RabbitMQ auf Seite iv.

5.3 Implementierung des HTTP-Clients

Um nach dem Empfangen eines Events die Synchronisierung in Richtung *HubSpot* anzustoßen wird ein HTTP-Client benötigt. Dafür wurde der *HubSpotApiClient* implementiert, welcher die zu sendenden Daten als Domänen-Model Parameter erhält, diese als Body setzt und damit einen HTTP request ausführt. Dieser Client ermöglicht POST und PATCH Anfragen bzw. das Erstellen und Bearbeiten von Kontaktdatensätzen in *HubSpot*.

Damit die Anfragen die korrekte *HubSpot* Instanz erreicht benötigt es eine *BaseUrl*, also den ersten Teil einer URL, wodurch die Instanz angesprochen wird. Desweiteren wird der Kontaktpfad benötigt, also

der Ort, an dem die Kontaktdaten verändert werden können. *HubSpot* benötigt für ihre Schnittstellen eine Authorisierung, weshalb der Authorization Header mit Bearer und einem Zugriffstoken für diese HTTP-Anfragen zusätzlich benötigt wird.

Um schließlich alle Anforderungen zu erfüllen wurde ein *HubSpotRateLimiter* implementiert, der dafür sorgt, dass die von *HubSpot* festgelegten Anfragebegrenzungen nicht überschritten werden und somit keine Anfragen unnötig fehlschlagen oder verloren gehen.

Ausschnitte zum *HubSpotApiClient* und *HubSpotRateLimiter* sind im Anhang A.5: HTTP-Client auf Seite viii zu finden.

5.4 Implementierung der Synchronisationshistorie

Um die Synchronisationshistorie persistieren zu können wurden drei Abstraktionsebenen entwickelt, um auf allen Ebenen testbar zu bleiben und die Aufteilung von Infrastruktur und Applikationsebene zu erhalten. Konkret handelt es sich dabei um *DbOperations*, *Repositories* und Services jeweils pro, den bereits genannten *SyncHistory* und *UserContactState*.

Die *DbOperations* befinden sich auf der niedrigsten Schicht und sie können einfache Abfragen auf Datenbankebene machen. Außerdem sind neben Testklassen, die einzigen Klassen, die den *DbContext* kennen.

Bei den *Repositories* handelt es sich, um Klassen, die *DbOperations* implementieren und neben diesen Aufrufen Domänen-Models zu Infrastruktur-Models konvertieren. Zudem befindet sich das Logging, Retry- und Errorhandling auf dieser Ebene. Für die Retrylogik wurde das NuGet Paket *Polly* verwendet, welches ermöglicht vorkonfigurierte *ResiliencePipelines* zu implementieren, mit denen man fehlerhafte Aufrufe für eine bestimmte Anzahl an Wiederholungen erneut ausführen kann.

Die Services beinhalten *Repository*-Aufrufe, sowie minimale Logik, mit der *Upserts* ermöglicht werden. So überprüft man für die *SyncHistory* zum Beispiel anhand der *EventId*, ob es bereits eine Historie gibt. Wenn es keine gibt wird eine erstellt, andernfalls wird sie nur aktualisiert.

Siehe Anhang A.6: Synchronisationshistorie auf Seite x für Code-Ausschnitte.

5.5 Implementierung der Geschäftslogik

Für die Umsetzung der eigentlichen Geschäftslogik wurden drei weitere Services auf Applikationsebene implementiert, damit auch hier die Testbarkeit und das *single responsibility principle* eingehalten wird. Zwei Services für die Ressourcen *PersonalInformation* und *CommunicationPreference* werden, je nach erhaltener Nachricht im EventConsumer instanziiert und starten den Synchronisationsprozess, der jeweilig deserialisierten Nachricht. Desweiteren beinhalten diese Services die Logik der Erstellung von Synchroinsationshistorien.

Als letztes wurde der *HubSpotContactService* entwickelt. Dieser beinhaltet den *HubSpotApiClient* und führt die beiden genannten Ressourcen ein *mapping* durch, womit diese in eine für *HubSpot* lesbare Form gebracht werden. Fakt ist, dass ein Kontakt nur erstellt werden soll, wenn die Ressource *PersonalInformation* das erste Mal synchronisiert wird. Aus diesem Grund wurden in dieser Klasse Methoden für das Erstellen und Aktualisieren von Kontakten implementiert. Wenn die *contactId*, also die Identifikationsnummer für einen *HubSpot* Kontakt null beträgt, wird ein Kontakt erstellt. Andernfalls wird der Kontakt mit der vorhandenen *contactId* aktualisiert. Für den Fall, dass ein Kontakt mit der gegebenen E-Mail bereits in *HubSpot* existiert, wurde ein *RegEx* eingebaut, welcher anhand des Inhalts, im *Json* Format einer API Antwort mit Statuscode 409, die existierende Indentifikationsnummer extrahiert und zurückgibt. Die *contactId* wird schließlich in dem *UserContactState* gespeichert, wodurch bei erneuter Datenänderung des Nutzers, nurnoch die Daten aktualisiert werden.

Siehe Anhang A.7: Geschäftslogik auf Seite xiii

6 Abnahmephase

Um zu versichern, dass Logik und Systemkomponenten wie gefordert funktionieren wurden der Applikation *Nightcrawler* Unit- sowie Integrationstests hinzugefügt. Welche ebenfalls bei jedem push in einen pull-request durch pipelines ausgeführt werden, um nach jeder Änderung zu gewährleisten, dass die Logik sich nicht geändert hat. Beim Framework zum Testen handelt es sich um *NSubstitute* mit *xUnit* und der Erweiterung *FluentAssertions*, was ermöglicht Tests in lesbarer Form im Arrange, Act und Assert Schema zu schreiben. Außerdem wurden die Tests immer nach dem Muster 'zu testende public Methode_Zustand_Resultat' benannt, um damit einheitliche und verständliche Tests zu gewährleisten.

6.1 Unittests

Pro .NET Projekt wurde ein neues Testprojekt für Unittests erstellt. Diese Art von Tests beschränken sich auf das Überprüfen von einzelnen Codeeinheiten. In diesem Fall wurde eine Unittestklasse pro zu testender Klasse erstellt. So erhält jede Testklasse eine echte Instanz der zu testenden Klasse, die aber mit substituierten/gemockten Abhängigkeiten instanziiert werden, sodass sich beim Testen rein auf die zu testende Logik beschränkt werden konnte.

6.2 Integrationstests

Integrationstest bilden den realen Ablauf von Programmen ab. Um echte Kommunikationswege testen zu können wurden die Pakete *Testcontainers.PostgreSql* und *Testcontainers.RabbitMQ* verwendet, die es ermöglichen Instanzen der jeweiligen Technologie mit Docker zu starten, welche in den Integrationstest, wie eine echte Instanz angesprochen werden können.

Im Anhang finden sich die Projektstruktur und Testbeispiele Anhang A.8: Tests auf Seite xvii.

6.3 Test Ergebnisse

Das Ergebnis dieser Tests ist das Erzielen einer Testabdeckung von 95 Prozent und einer fehlerfreien Anwendung.

Symbol	Coverage (%)	Uncovered/Total Stmts.
Total	95%	112/2471
Infrastructure.PostgreSql	100%	0/156
Domain	100%	0/47
Tests	98%	20/1257
Application	95%	12/254
IntegrationTests	94%	11/190
Infrastructure.PostgreSql.Migrations	92%	13/153
Infrastructure.RabbitMQ	91%	13/140
Presentation.RabbitMQ	87%	24/185
Infrastructure.HubSpot	82%	15/85
Orchestrator	0%	4/4

Anhang A.8: Tests auf Seite xvii

7 Deployment

Rahmenbedingung des Projektes ist nicht die Aufnahme der Software in den Livebetrieb, jedoch in das Testsystem. Aus diesem Grund wurden GitHub Pipelines implementiert, die alle Testpfade und eine statische Codeanalyse mit *SonarQube* ausführen und bei erfolgreicher Reviewphase eines pull-requests, die Änderungen automatisiert in den main branch des GitHub Repositories mergen. Gleichzeitig bedeutet dies, dass die Software auf einem Linux Debian basierten Server deployed und gehosted wird. Denn die Pipelines führen, nach bestehen aller Tests und des SonarQube Quality Gates, ein Deployment zu dem Testserver über SSH aus.

Im Anhang sind Ausschnitte der yml Dateien zu finden Anhang A.9: Pipelines und Automatisierung auf Seite xx

8 Dokumentation

In diesem Abschnitt wird auf die im Rahmen des Projektes erstellte Entwicklerdokumentation eingegangen.

8.1 Entwicklerdokumentation

Da die Anwendung nicht direkt von Nutzern ansprechbar ist benötigt sie keine Anwenderdokumentation. Lediglich Entwickler müssen wissen, wie man die Anwendung verwendet. Aus diesem Grund wurde eine Entwicklerdokumentation in Form einer README.md erstellt.

Die README.md Datei enthält eine detaillierte Anleitung für die Nutzung der Anwendung bestehend aus den Voraussetzungen für das erfolgreiche starten, einer Quick Start Anleitung, der Projektstruktur, einer Auflistung aller verwendeten Konfigurationseinstellungen und Umgebungsvariablen, sowie einer Anleitung für das Starten der Applikation mit Docker Compose.

Ein Ausschnitt aus der erstellten Benutzerdokumentation befindet sich im Anhang A.10: Entwicklerdokumentation auf Seite xxiv.

9 Fazit

In diesem Abschnitt geht es darum den Abschluss des Projektes sachlich zu bewerten und transparent darzustellen, inwiefern die Projektplanung eingehalten werden konnte. Außerdem wird gezeigt, was der Autor gelernt hat und wie sich das im Projekt behandelte System in Zukunft weiterentwickeln wird.

9.1 Soll-/Ist-Vergleich

Das Projektziel die dezentralisierte Verfügbarkeit von Daten zweier Kundendaten speichernder Systeme aufzulösen und somit eine Source of Truth zu schaffen wurde erreicht. Die Funktionalität und Einhaltung der Anforderungen wurde somit erfüllt.

Bei der Einhaltung der Projektplanung ist es im Hinblick auf Zeit und Kosten zu Abweichungen gekommen. So wurde das Zeitlimit, um fünf Stunden überschritten und die Kosten sind dazu proportional um 175 Euro gestiegen. Dies liegt daran, dass die Nutzung der Technologie *RabbitMQ* und den einhergehenden Voraussetzungen im Unternehmen bisher wenig erforscht war. Weshalb dieses Projekt als ein Beispiel für die zukünftige Nutzung von Message Brokern und Event Queues dienen wird. Aufgrund der Unerfahrenheit ist es dementsprechend bei der Programmierung und der Reviewphase zu Verzögerungen gekommen.

Wie in Tabelle 3 zu erkennen ist, konnte die Zeitplanung nicht eingehalten werden.

Phase	Geplant	Tatsächlich	Differenz
Analyse	8 h	8 h	
Entwurf	6 h	6 h	
Implementierung	49 h	54 h	+5 h
Deployment	7 h	7 h	
Dokumentation	10 h	10 h	
Gesamt	80 h	85 h	

Tabelle 3: Soll-/Ist-Vergleich

9.2 Lessons Learned

Der Prüfling hat bei der Durchführung des Projektes gelernt realistischere Einschätzungen zu machen in Bezug auf die Zeitplanung bei bisher unbekannten Technologien. Außerdem ist dem Prüfling nun der Umgang mit *RabbitMQ*, den einhergehenden Topologien und Einstellungen für Queues vertraut. Zudem kann der Prüfling zukünftlich für Fragen in diesem Bereich zur Seite stehen und anderen Entwicklern bei der Implementierung helfen.

9.3 Ausblick

Im Rahmen des Projektes wurde sich bei der zu übertragenen Datenvielfalt zurückgehalten. Zukünftlich soll also eine Erweiterung implementiert werden, bei der weitere kundenspezifische Daten synchronisiert werden. Des Weiteren könnte man zur Kundendatenabfrage eine REST-API einbauen, welche es ermöglicht, die zentralisiert gespeicherten Daten auch in anderen Bereichen zu erfragen.

Literaturverzeichnis

Eidesstattliche Erklärung

Ich, Niklas Hamburg, versichere hiermit, dass ich meine **Dokumentation zur betrieblichen Projektarbeit** mit dem Thema

> *Bereitstellung eines Microservices – zur Synchronisation von dezentralisiert gespeicherten Kundendaten*

selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, wobei ich alle wörtlichen und sinngemäßen Zitate als solche gekennzeichnet habe. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Papenburg, den 08.05.2026

Niklas Hamburg

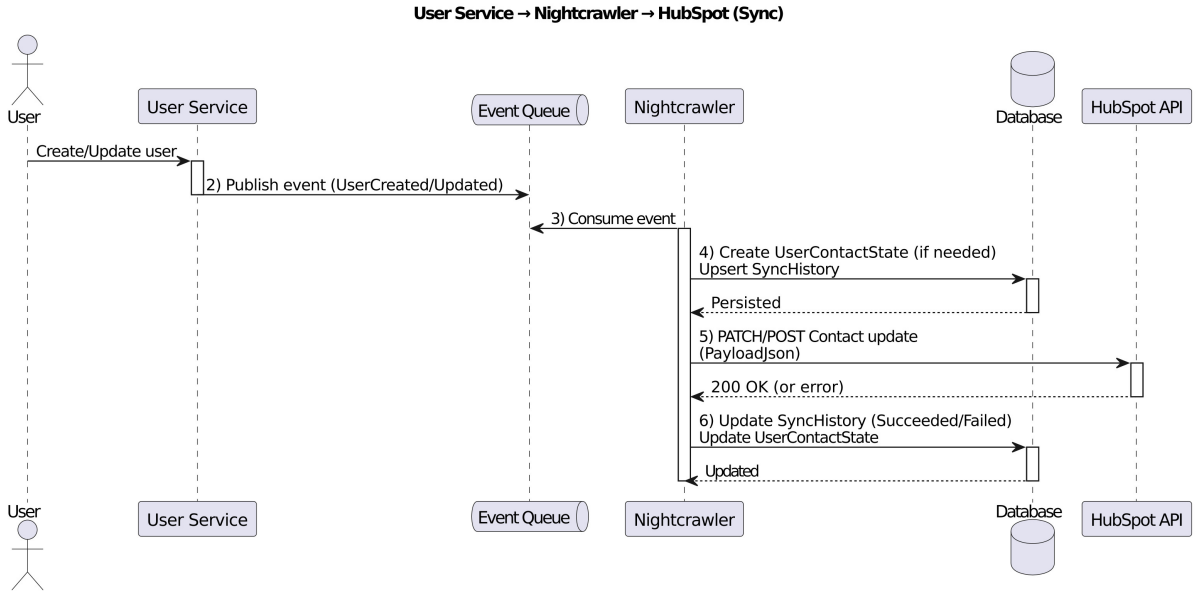
A Anhang

A.1 Detaillierte Zeitplanung

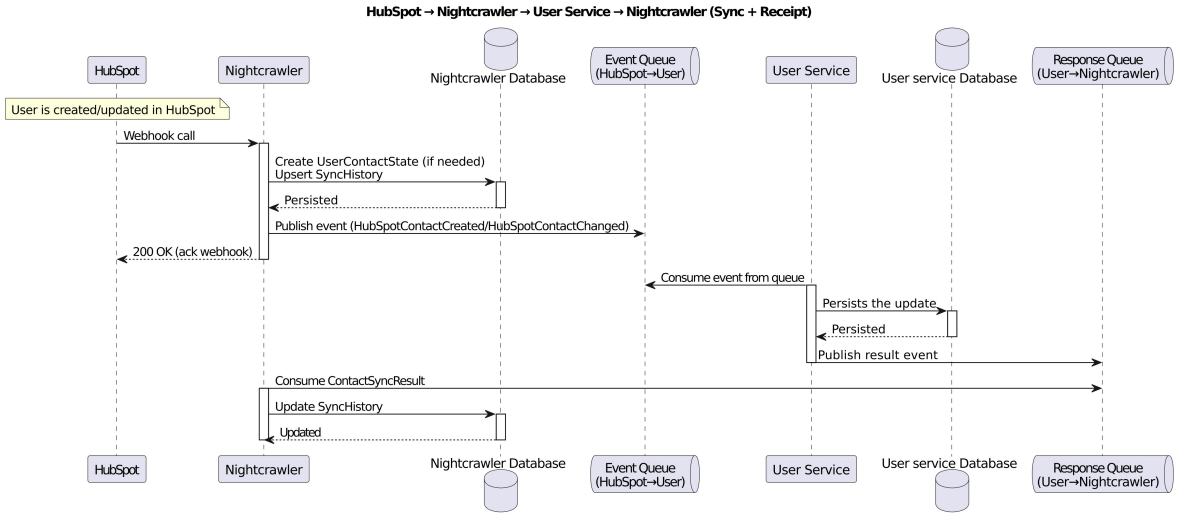
Analyse		**8 h**
1. Ist-Analyse durchführen	2 h	
2. Wirtschaftlichkeitsprüfung	2 h	
3. Anforderungsanalyse	1 h	
4. Nutzwertanalyse	3 h	
Entwurf		**6 h**
1. UML-Aktivitätsdiagramm	2 h	
2. Architekturübersicht	2 h	
3. Entwurf von Datenbankstruktur und Tabellen	2 h	
Implementierung		**49 h**
1. Repo/Projektsetup	1 h	
2. HTTP-Clients	7 h	
3. Authentifizierung	1 h	
4. Anbindung Hangfire	5 h	
5. Implementierung Sync-Service	2 h	
5.1. Implementierung Sync-Repository	10 h	
5.2. Implementierung Sync-DbOperations	4 h	
5.3. Implementierung Hangfire Backgroundjobs	3 h	
6. Mapping/Validierung	1 h	
7. Anbindung/Erstellung Datenbank	3 h	
8. Errorhandling/Logging/Retry	5 h	
9. Unit-/Integrationstests	7 h	
Deployment		**7 h**
1. Einbindung Dockerfiles	1 h	
2. Konfiguration GitHub Secrets/Variables	1 h	
3. CI/CD Pipelines	4 h	
4. Deployment CSharp-Microservice	1 h	
Dokumentation		**10 h**
1. Erstellen der Projektdokumentation	8 h	
2. Erstellen der Entwicklerdokumentation	2 h	
Gesamt		**80 h**

A.2 Synchronisations Abläufe

User-Service zu HubSpot



HubSpot zu User-Service



A.3 Datenbankmodell

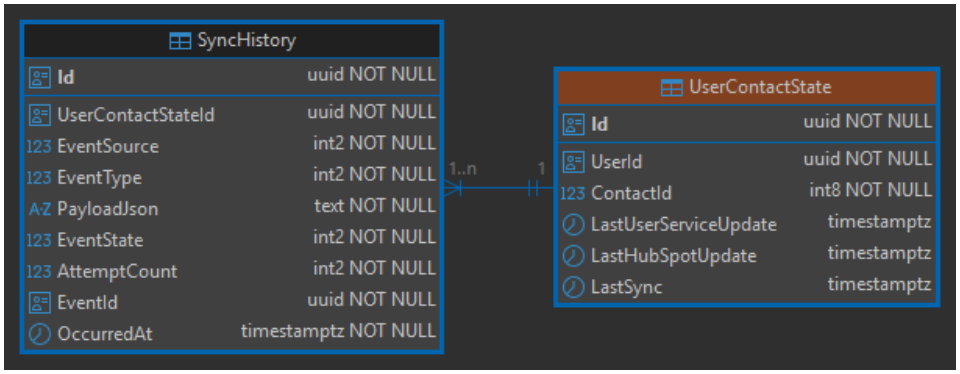


Abbildung 1: Nightcrawler ER-Diagramm

A.4 RabbitMQ

```
private static IServiceCollection AddRabbitMqConnectionDependencies(this IServiceCollection services)
{
    services.AddSingleton<IConnectionFactory>(serviceProvider =>
    {
        ConnectionOptions connectionOptions = serviceProvider.GetRequiredService<IOptions<ConnectionOptions>>().Value;

        ConnectionFactory connectionFactory = new()
        {
            HostName = connectionOptions.HostName,
            Port = connectionOptions.Port,
            UserName = connectionOptions.UserName,
            Password = connectionOptions.Password
        };

        return connectionFactory;
    });

    return services;
}
```

Abbildung 2: RabbitMQ DI Registrierung

```
public async Task StartAsync(CancellationToken cancellationToken)
{
    try
    {
        await using IConnection connection =
            await _connectionFactory.CreateConnectionAsync(cancellationToken);

        await using IChannel channel =
            await connection.CreateChannelAsync(cancellationToken: cancellationToken);

        await channel.ExchangeDeclareAsync(
            exchange: _queueOptions.ExchangeName,
            type: ExchangeType.Direct,
            durable: true,
            autoDelete: false,
            cancellationToken: cancellationToken); // Task

        await DeclareMainQueueAsync(channel, cancellationToken);
        await BindQueueAsync(
            channel,
            _queueOptions.QueueNameMain,
            _queueOptions.MainRoutingKey,
            cancellationToken);

        await DeclareRetry30SQueueAsync(channel, cancellationToken);
        await DeclareRetry1MQueueAsync(channel, cancellationToken);

        await DeclareDeadLetterQueueAsync(channel, cancellationToken);
        await BindQueueAsync(
            channel,
            _queueOptions.QueueNameDlq,
            _queueOptions.DeadRoutingKey,
            cancellationToken);

        _logger.LogInformation("Queues initialized successfully!");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, message: "An error occurred while initializing the queues.");
    }
}
```

Abbildung 3: Queue Initialisierung Hauptmethode

```
public async Task PublishAsync<TPayload>(
    EventMessage<TPayload> message,
    CancellationToken cancellationToken)
{
    string serializedMessage = JsonSerializer.Serialize(message);
    byte[] body = Encoding.UTF8.GetBytes(serializedMessage);

    BasicProperties properties = new()
    {
        Persistent = true,
        ContentType = "application/json",
        Type = Enum.GetName(message.EventSubject),
        MessageId = message.EventId.ToString()
    };

    await _lock.WaitAsync(cancellationToken);
    try
    {
        await _resiliencePipeline.ExecuteAsync(
            async ct => await PublishInternalAsync(properties, body, ct),
            cancellationToken); // ValueTask

        if (_logger.IsEnabled(LogLevel.Information))
        {
            _logger.LogInformation(
                "Successfully published message {MessageId} of type {Type} to exchange {Exchange} with routing key {RoutingKey}",
                properties.MessageId,
                properties.Type,
                _queueOptions.ExchangeName,
                _queueOptions.MainRoutingKey);
        }
    }
    catch (PublishException ex) when (ex.IsReturn)
    {
        _logger.LogError(
            ex,
            message: "Message {MessageId} of type {Type} was returned by broker and will not be retried. Exchange: {Exchange}, RoutingKey: {RoutingKey}",
            properties.MessageId,
            properties.Type,
            _queueOptions.ExchangeName,
            _queueOptions.MainRoutingKey);

        throw new EventPublishException(
            $"Publish failed for message {properties.MessageId} of type {properties.Type}", ex);
    }
    catch (Exception ex)
    {
        _logger.LogError(
            ex,
            message: "Failed to publish message {MessageId} of type {Type}.",
            properties.MessageId,
            properties.Type);

        throw new EventPublishException(
            $"Publish failed for message {properties.MessageId} of type {properties.Type}", ex);
    }
    finally
    {
        _lock.Release();
    }
}
```

Abbildung 4: Event Publisher Hauptmethode

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    await EnsureInitializedAsync(stoppingToken);

    await _channel!.BasicQosAsync(
        prefetchSize: 0,
        prefetchCount: 1,
        global: false,
        cancellationToken: stoppingToken); // Task

    AsyncEventingBasicConsumer consumer = new(_channel);
    consumer.ReceivedAsync += async (_, eventArgs)
        => await OnReceivedAsync(eventArgs, stoppingToken);

    _consumerTag = await _channel.BasicConsumeAsync(
        queue: _queueOptions.QueueNameMain,
        autoAck: false,
        consumer: consumer,
        cancellationToken: stoppingToken); // Task<string>

    if (_logger.IsEnabled(LogLevel.Information))
    {
        _logger.LogInformation(
            "Started consuming queue {QueueName} with consumer tag {ConsumerTag}",
            _queueOptions.QueueNameMain,
            _consumerTag);
    }

    try
    {
        await Task.Delay(Timeout.Infinite, stoppingToken);
    }
    catch (OperationCanceledException ex)
    {
        if (_logger.IsEnabled(LogLevel.Information))
        {
            _logger.LogInformation(
                ex,
                message: "Stopping RabbitMQ consumer for queue {QueueName}.",
                _queueOptions.QueueNameMain);
        }
    }
}
```

Abbildung 5: Event Consumer Hauptmethode

A.5 HTTP-Client

```
public class HubSpotApiClient : IHubSpotApiClient
{
    private const string s_contactsPath = "/crm/v3/objects/contacts";

    private readonly HttpClient _httpClient;
    private readonly IHubSpotRateLimiter _rateLimiter;
    private readonly IJsonContentMapper _mapper;

    1 usage   Nhamburg Journaway
    public HubSpotApiClient(
        IHttpClientFactory httpClientFactory,
        IHubSpotRateLimiter rateLimiter,
        IOptions<HubSpotOptions> options,
        IJsonContentMapper mapper)
    {
        HubSpotOptions hubSpotOptions = options.Value;
        _httpClient = httpClientFactory.CreateClient(nameof(HubSpotApiClient));
        _rateLimiter = rateLimiter;
        _mapper = mapper;
        _httpClient.BaseAddress = new Uri(hubSpotOptions.BaseUrl);
        _httpClient.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue( scheme: "Bearer", hubSpotOptions.AccessToken);
    }

    2+12 usages   2 tests OK   Nhamburg Journaway
    public async Task<HttpResponseMessage> PostAsync(HubSpotPostRequest requestBody, CancellationToken cancellationToken)
    {
        await _rateLimiter.WaitForAvailabilityAsync(cancellationToken);

        using HttpRequestMessage requestMessage = new(
            HttpMethod.Post,
            requestUri: s_contactsPath);
        requestMessage.Content = _mapper.ToJsonContent(requestBody);

        HttpResponseMessage response = await _httpClient.SendAsync(requestMessage, cancellationToken);

        _rateLimiter.UpdateFromHeaders(response.Headers);
        return response;
    }

    1+13 usages   Nhamburg Journaway
    public async Task<HttpResponseMessage> PatchAsync(HubSpotPatchRequest requestBody, long contactId, CancellationToken cancellationToken)
    {
        await _rateLimiter.WaitForAvailabilityAsync(cancellationToken);

        string path = $"{s_contactsPath}/{contactId}";

        using HttpRequestMessage requestMessage = new(HttpMethod.Patch, path);
        requestMessage.Content = _mapper.ToJsonContent(requestBody);

        HttpResponseMessage response = await _httpClient.SendAsync(requestMessage, cancellationToken);

        _rateLimiter.UpdateFromHeaders(response.Headers);
        return response;
    }
}
```

Abbildung 6: HubSpotApiClient

```
public async Task WaitForAvailabilityAsync(CancellationToken cancellationToken)
{
    while (true)
    {
        TimeSpan delay;

        await _gate.WaitAsync(cancellationToken);
        try
        {
            DateTimeOffset now = DateTimeOffset.UtcNow;
            TrimExpired(now);

            if (_requestTimestamps.Count < _maxRequestsPerWindow)
            {
                _requestTimestamps.Enqueue(now);
                return;
            }

            DateTimeOffset oldest = _requestTimestamps.Peek();
            DateTimeOffset nextAllowed = oldest + _window;
            delay = nextAllowed - now;
        }
        finally
        {
            _gate.Release();
        }

        if (_logger.IsEnabled(LogLevel.Debug))
        {
            _logger.LogDebug(
                "HubSpot throttling for {DelayMs} ms (maxRequests={MaxRequests}, windowMs={WindowMs})",
                (int)delay.TotalMilliseconds,
                _maxRequestsPerWindow,
                (int)_window.TotalMilliseconds);
        }

        await Task.Delay(delay, cancellationToken);
    }
}

2+5 usages   Nhamburg Journaway
public void UpdateFromHeaders(HttpResponseHeaders headers)
{
    if (!TryGetInt(headers, name: s_hubspotRateLimitMax, out int max) ||
        !TryGetInt(headers, name: s_hubspotRateLimitIntervalMilliseconds, out int intervalMs))
    {
        return;
    }

    if (max <= 0 || intervalMs <= 0)
    {
        return;
    }

    _maxRequestsPerWindow = max;
    _window = TimeSpan.FromMilliseconds(intervalMs);
}
```

Abbildung 7: HubSpotApiRateLimiter

A.6 Synchronisationshistorie

```
namespace Infrastructure.PostgreSql.DbOperations;

5 usages  Nhamburg Journaway
public class SyncHistoryDbOperations : ISyncHistoryDbOperations
{
    private readonly ApplicationDbContext _dbContext;

    8 usages  4 tests OK  Nhamburg Journaway
    public SyncHistoryDbOperations(ApplicationDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    1+7 usages  1 test OK  Nhamburg Journaway
    public async Task<SyncHistoryDto> SingleByEventIdAsync(Guid eventId, CancellationToken cancellationToken)
    {
        return await _dbContext.SyncHistories // DbSet<SyncHistoryDto>
            .SingleAsync(syncHistory => syncHistory.EventId == eventId, cancellationToken); // Task<SyncHistoryDto>
    }

    1+2 usages  1 test OK  Nhamburg Journaway
    public void Add(SyncHistoryDto syncHistory)
    {
        _dbContext.Add(syncHistory);
    }

    1+3 usages  1 test OK  Nhamburg Journaway
    public void Update(SyncHistoryDto syncHistory, SyncHistoryDto updatedSyncHistory)
    {
        _dbContext.Entry(syncHistory).CurrentValues.SetValues(updatedSyncHistory);
    }

    1+6 usages  1 test OK  Nhamburg Journaway
    public async Task<int> SaveChangesAsync(CancellationToken cancellationToken)
    {
        return await _dbContext.SaveChangesAsync(cancellationToken);
    }
}
```

Abbildung 8: DbOperation SyncHistory

```
public async Task<SyncHistoryModel?> GetAsync(EventId eventId, CancellationToken cancellationToken)
{
    cancellationToken.ThrowIfCancellationRequested();

    SyncHistoryDto syncHistoryDto;
    try
    {
        syncHistoryDto = await FindByEventIdAsync(eventId, cancellationToken);
    }
    catch (SyncHistoryNotFoundException)
    {
        return null;
    }

    SyncHistoryModel syncHistoryModel = _mapper.ToDomain(syncHistoryDto);
    return syncHistoryModel;
}

2+4 usages  2 tests OK  Nhamburg Journaway
public async Task CreateAsync(SyncHistoryModel syncHistoryModel, CancellationToken cancellationToken)
{
    cancellationToken.ThrowIfCancellationRequested();

    Guid syncHistoryId = Guid.Empty;
    SyncHistoryDto syncHistoryDto = _mapper.ToDto(syncHistoryModel, syncHistoryId);

    _dbOperations.Add(syncHistoryDto);

    await SaveChangesAsync(syncHistoryModel.EventId, cancellationToken);
}

3+4 usages  3 tests OK  Nhamburg Journaway
public async Task UpdateAsync(SyncHistoryModel syncHistoryModel, CancellationToken cancellationToken)
{
    cancellationToken.ThrowIfCancellationRequested();

    SyncHistoryDto syncHistoryDto =
        await FindByEventIdAsync(syncHistoryModel.EventId, cancellationToken);
    SyncHistoryDto updatedSyncHistoryDto =
        _mapper.ToDto(syncHistoryModel, syncHistoryDto.Id);

    _dbOperations.Update(syncHistoryDto, updatedSyncHistoryDto);

    await SaveChangesAsync(syncHistoryModel.EventId, cancellationToken);
}

2 usages  5 tests OK  Nhamburg Journaway
private async Task<SyncHistoryDto> FindByEventIdAsync(EventId eventId, CancellationToken cancellationToken)
{
    try
    {
        return await _dbOperations.SingleByEventIdAsync(eventId.Value, cancellationToken);
    }
    catch (InvalidOperationException ex)
    {
        _logger.LogWarning(ex, message: "SyncHistory with EventId {EventId} could not be found: {Message}",
            eventId.Value, ex.Message);

        throw new SyncHistoryNotFoundException(
            $"SyncHistory with EventId {eventId.Value} could not be found: {ex.Message}", ex);
    }
}

2 usages  4 tests OK  Nhamburg Journaway
private async Task SaveChangesAsync(EventId eventId, CancellationToken cancellationToken)
{
    try
    {
        await _resiliencePipeline.ExecuteAsync(
            async ct => await _dbOperations.SaveChangesAsync(ct),
            cancellationToken); //ValueTask<int>
    }
    catch (Exception ex) when (ex is DbUpdateException or DbUpdateConcurrencyException)
    {
        _logger.LogError(ex,
            message: "An error occurred while updating a syncHistory with EventId: {EventId} in the database: {Message}",
            eventId.Value, ex.Message);

        throw new SyncHistoryDbUpdateException(
            $"An error occurred while updating a syncHistory with EventId: {eventId.Value} in the database: {ex.Message}",
            ex);
    }
}
```

Abbildung 9: Repository SyncHistory

```
namespace Application.Services;

3 usages  Nhamburg Journaway
public class SyncHistoryService : ISyncHistoryService
{
    private readonly ISyncHistoryRepository _repository;

    1 usage  4 tests OK  Nhamburg Journaway
    public SyncHistoryService(ISyncHistoryRepository repository)
    {
        _repository = repository;
    }

    4+20 usages  4 tests OK  Nhamburg Journaway
    public async Task UpsertAsync(
        SyncHistoryModel syncHistory,
        CancellationToken cancellationToken)
    {
        SyncHistoryModel? model = await _repository.GetAsync(syncHistory.EventId, cancellationToken);
        if (model is null)
        {
            await _repository.CreateAsync(syncHistory, cancellationToken);
            return;
        }

        await _repository.UpdateAsync(syncHistory, cancellationToken);
    }
}
```

Abbildung 10: Service SyncHistory

A.7 Geschäftslogik

```
9+7 usages  Nhamburg Journaway
public async Task<long> SyncPersonalInformationAsync(
    PersonalInformationChangedPayload payload,
    long contactId,
    CancellationToken cancellationToken)
{
    ArgumentNullException.ThrowIfNull(payload);

    Dictionary<string, object?> properties = BuildPersonalInformationProperties(payload);

    if (contactId is 0)
    {
        return await CreateContactAsync(properties, cancellationToken);
    }

    await UpdateContactAsync(contactId, properties, cancellationToken);

    return contactId;
}
```

Abbildung 11: PersonalInformation Upsert Logik

```
private async Task<long> CreateContactAsync(
    Dictionary<string, object?> properties,
    CancellationToken cancellationToken)
{
    string traceId = Guid.NewGuid().ToString();

    HubSpotPostRequest requestBody = new()
    {
        Properties = properties,
        ObjectWriteTraceId = traceId
    };

    using HttpResponseMessage response = await _apiClient.PostAsync(requestBody, cancellationToken);
    string responseJson = await response.Content.ReadAsStringAsync(cancellationToken);

    if (response.StatusCode == HttpStatusCode.Conflict)
    {
        long existingContactId = ExtractExistingContactIdFromConflictResponse(responseJson);

        if (_logger.IsEnabled(LogLevel.Information))
        {
            _logger.LogInformation(
                "HubSpot contact already exists. Updating existing contact (contactId={ContactId}, traceId={TraceId})",
                existingContactId,
                traceId);
        }

        await UpdateContactAsync(existingContactId, properties, cancellationToken);

        return existingContactId;
    }

    if (response.StatusCode != HttpStatusCode.Created)
    {
        _logger.LogError(
            "HubSpot create contact failed (statusCode={StatusCode}, traceId={TraceId}, response={Response})",
            (int)response.StatusCode,
            traceId,
            responseJson);

        throw new HubSpotSyncException("HubSpot create contact failed");
    }

    HubSpotPostResponse? parsed =
        JsonSerializer.Deserialize<HubSpotPostResponse>(responseJson, s_jsonOptions);

    if (parsed is not null && long.TryParse(parsed.Id, out long contactId))
    {
        if (_logger.IsEnabled(LogLevel.Information))
        {
            _logger.LogInformation(
                "HubSpot create contact succeeded (contactId={ContactId}, traceId={TraceId})",
                contactId,
                traceId);
        }

        return contactId;
    }

    _logger.LogError(
        "ContactId could not be retrieved from response (traceId={TraceId})",
        traceId);

    throw new HubSpotSyncException("HubSpot ContactId could not be retrieved from response");
}
```

Abbildung 12: API Logik

```
private static long ExtractExistingContactIdFromConflictResponse(string responseJson)
{
    Match match = s_existingContactIdRegex.Match(responseJson);

    if (match.Success && long.TryParse(match.Groups["id"].Value, out long contactId))
    {
        return contactId;
    }

    throw new HubSpotSyncException(
        "HubSpot contact already exists, but ContactId could not be extracted from conflict response");
}

[GeneratedRegex( pattern: """Existing ID:\s*"?(?<id>\d+)"?""", options: RegexOptions.IgnoreCase | RegexOptions.Compiled | RegexOptions.CultureInvariant, matchTimeoutMilliseconds: 100)]
1 usage   Nhamburg Journaway
private static partial Regex ExistingContactIdRegex();
```

Abbildung 13: RegEx Aufruf

```
public async Task SyncAsync(
    EventMessage<CommunicationPreferenceChangedPayload> message,
    short attempt,
    CancellationToken cancellationToken)
{
    UserId userId = new(message.Payload.UserId);

    UserContactStateModel userContactState =
        await GetUserContactStateAsync(userId, cancellationToken);

    string payloadJson = JsonSerializer.Serialize(message.Payload, s_jsonOptions);
    SyncHistoryId id = new(Guid.NewGuid());
    EventId eventId = new(message.EventId);

    SyncHistoryModel syncHistory = new(
        Id: id,
        UserContactStateId: userContactState.Id,
        EventId: eventId,
        EventSource: message.EventSource,
        EventType: message.EventType,
        PayloadJson: payloadJson,
        OccurredAt: message.OccurredAt.UtcDateTime,
        EventState: EventState.Pending,
        AttemptCount: attempt);
    try
    {
        bool isCreated = await UpsertIgnoredSyncHistoryAsync(syncHistory, userContactState, cancellationToken);
        if (isCreated)
        {
            return;
        }

        await UpsertPendingSyncHistoryAsync(syncHistory, cancellationToken);

        await _hubSpotContactService.SyncCommunicationPreferenceAsync(message.Payload, userContactState.ContactId, cancellationToken);

        await UpdateUserContactStateAsync(message.OccurredAt, userContactState, cancellationToken);

        await UpdateSucceededSyncHistoryAsync(syncHistory, cancellationToken);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, message: "Unexpected error occured while processing '{EventId}'", message.EventId);

        await UpsertFailedSyncHistoryAsync(syncHistory, cancellationToken);

        throw new CommunicationPreferenceSyncException(
            $"An error occurred while syncing CommunicationPreference {ex.Message}", ex);
    }
}
```

Abbildung 14: CommunicationPreference Logik

A.8 Tests

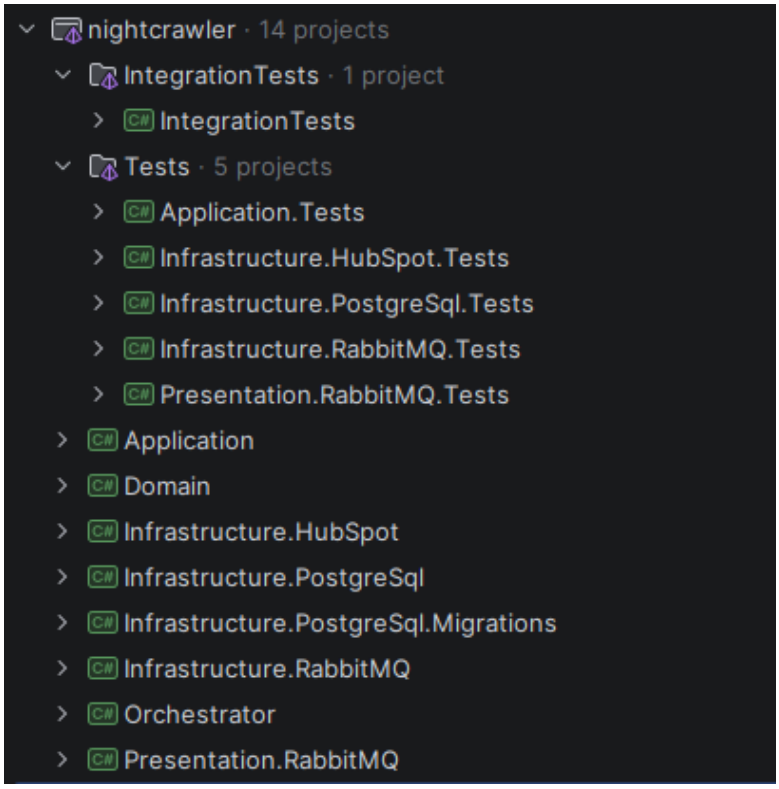


Abbildung 15: Nightcrawler Projekt Struktur

Symbol	Coverage (%)	Uncovered/Total Stmts.
Total	95%	112/2471
Infrastructure.PostgreSql	100%	0/156
Domain	100%	0/47
Tests	98%	20/1257
Application	95%	12/254
IntegrationTests	94%	11/190
Infrastructure.PostgreSql.Migrations	92%	13/153
Infrastructure.RabbitMQ	91%	13/140
Presentation.RabbitMQ	87%	24/185
Infrastructure.HubSpot	82%	15/85
Orchestrator	0%	4/4

Abbildung 16: Testabdeckung

```
[Fact]
1 test OK   Nhamburg Journaway
public async Task SyncCommunicationPreferenceAsync_WithValidPayload_UpdatesContact()
{
    // Arrange
    CancellationToken cancellationToken = CancellationToken.None;

    CommunicationPreferenceChangedPayload payload = CreateCommunicationPreferencePayload(
        isNewsletterEnabled: true);

    _apiClient // IHubSpotApiClient
        .PatchAsync(
            requestBody: Arg.Any<HubSpotPatchRequest>(),
            contactId: Arg.Any<long>(),
            cancellationToken: Arg.Any<CancellationToken>()) // Task<HttpResponseMessage>
        .Returns( returnThis: CreateResponse(HttpStatusCode.OK));

    // Act
    await _sut.SyncCommunicationPreferenceAsync(
        payload,
        contactId: 123,
        cancellationToken); // Task

    // Assert
    await _apiClient.Received(1).PatchAsync(
        requestBody: Arg.Is<HubSpotPatchRequest>(request =>
            request.Properties["ja_userid"]!.Equals(payload.UserId.ToString()) &&
            request.Properties["agreement_newsletter"]!.Equals(true) &&
            !string.IsNullOrWhiteSpace(request.ObjectWriteTraceId)),
        contactId: 123,
        cancellationToken); // Task<HttpResponseMessage>
}
```

Abbildung 17: Unit Test

```
[Fact]
1 test OK   Nhamburg Journaway
public async Task Add_SyncHistoryWasAdded_DbContextContainsAddedSyncHistory()
{
    // Arrange
    Guid syncHistoryId = Guid.NewGuid();
    Guid userContactStateId = Guid.NewGuid();

    SyncHistoryDto syncHistoryDto = new()
    {
        Id = syncHistoryId,
        UserContactStateId = userContactStateId,
        EventId = Guid.NewGuid(),
        EventSource = EventSource.UserService,
        EventType = EventType.Create,
        PayloadJson = """{"email":"test@example.com"}""",
        OccurredAt = DateTime.UtcNow,
        EventState = EventState.Pending,
        AttemptCount = 1
    };

    await SeedUserContactStateAsync(userContactStateId, CancellationToken.None);

    await using ApplicationDbContext dbContext = _factory.CreateDbContext();
    SyncHistoryDbOperations sut = new(dbContext);

    // Act
    sut.Add(syncHistoryDto);

    // Assert
    dbContext.SyncHistories.Local.Should() // GenericCollectionAssertions<SyncHistoryDto>
        .ContainSingle(syncHistory => syncHistory == syncHistoryDto);
}
```

Abbildung 18: Integrationstest

A.9 Pipelines und Automatisierung

```yaml
jobs:
  run-all-tests:
    steps:
      - name: SonarQube Scan & Run tests
        shell: bash
        env:
          SONAR_TOKEN: ${{ secrets.sonar_token }}
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          REPOSITORY_OWNER: ${{ github.repository_owner }}
          REPOSITORY_NAME: ${{ github.event.repository.name }}
          HEAD_REF: ${{ github.head_ref }}
          BASE_REF: ${{ github.base_ref }}
          REF_NAME: ${{ github.ref_name }}
          PR_NUMBER: ${{ github.event.pull_request.number }}
        run: |
          set -Eeuo pipefail
          IFS=$'\n\t'
          set -f

          SONAR_SCANNER="./.sonar/scanner/dotnet-sonarscanner"

          EXTRA_ARGS=()

          if [[ "${GITHUB_EVENT_NAME}" == "pull_request" ]]; then
            EXTRA_ARGS+=("/d:sonar.pullrequest.base=${BASE_REF}")
            EXTRA_ARGS+=("/d:sonar.pullrequest.branch=${HEAD_REF}")
            EXTRA_ARGS+=("/d:sonar.pullrequest.key=${PR_NUMBER}")
          else
            EXTRA_ARGS+=("/d:sonar.branch.name=${REF_NAME}")
          fi

          dotnet sonarscanner begin \
            /k:"${REPOSITORY_OWNER}_${REPOSITORY_NAME}" \
            /n:"${REPOSITORY_NAME}" \
            /o:"${REPOSITORY_OWNER}" \
            /d:sonar.token="${SONAR_TOKEN}" \
            /d:sonar.host.url="https://sonarcloud.io" \
            /d:sonar.cs.opencover.reportsPaths=".coverage/**/coverage.opencover.xml" \
            /d:sonar.cs.vstest.reportsPaths=".testresults/**/*.trx" \
            /d:sonar.scanner.skipJreProvisioning=true \
            "${EXTRA_ARGS[@]}"

          dotnet restore --locked-mode --nologo
          dotnet build --no-restore --nologo

          mkdir -p .coverage
          dotnet test \
            --no-build \
            --nologo \
            --logger:"trx" \
            --results-directory .testresults \
            /p:CollectCoverage=true \
            /p:CoverletOutput="../.coverage/" \
            /p:MergeWith="../.coverage/coverage.json" \
            /p:CoverletOutputFormat=\"json,opencover\"

          dotnet sonarscanner end /d:sonar.token="${SONAR_TOKEN}"
```

Abbildung 19: Tests und statische Codeanalyse

```yaml
build-and-publish-image:
  needs: [ run-all-tests ]
  runs-on: ubuntu-latest
  permissions:
    contents: write
    packages: write
  outputs:
    version: ${{ steps.semver.outputs.version }}
  steps:
    - name: Checkout Code
      uses: actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd #v6.0.2
      with:
        fetch-depth: 0

    - name: Generate semantic version
      id: semver
      uses: paulhatch/semantic-version@a8f8f59fd7f0625188492e945240f12d7ad2dca3 #v5.4.0
      with:
        tag_prefix: "v"
        major_pattern: "/!:|BREAKING CHANGE:/"
        minor_pattern: "/feat:/"
        version_format: "v${major}.${minor}.${patch}-beta.${increment}"

    - name: Build and push Orchestrator Docker image
      uses: ./.github/actions/build-docker-image
      with:
        image_name: ${{ vars.DOCKER_IMAGE_ORCHESTRATOR }}
        dockerfile: ./Orchestrator/Dockerfile
        context: .
        push: true
        version_tag: ${{ steps.semver.outputs.version }}
        docker_registry: ${{ vars.DOCKER_REGISTRY }}
        registry_user: ${{ github.actor }}
        registry_owner: ${{ github.repository_owner }}
        registry_password: ${{ secrets.GITHUB_TOKEN }}

    - name: Build and push Infrastructure.PostgreSql.Migrations Docker image
      uses: ./.github/actions/build-docker-image
      with:
        image_name: ${{ vars.DOCKER_IMAGE_MIGRATION }}
        dockerfile: ./Infrastructure.PostgreSql.Migrations/Dockerfile
        context: .
        push: true
        version_tag: ${{ steps.semver.outputs.version }}
        docker_registry: ${{ vars.DOCKER_REGISTRY }}
        registry_user: ${{ github.actor }}
        registry_owner: ${{ github.repository_owner }}
        registry_password: ${{ secrets.GITHUB_TOKEN }}
```

Abbildung 20: Docker Images vorbereiten

```yaml
      - name: Create and push Git tag
        shell: bash
        env:
          VERSION: ${{ steps.semver.outputs.version }}
        run: |
          set -Eeuo pipefail
          IFS=$'\n\t'
          set -f

          if git rev-parse -q --verify "refs/tags/$VERSION" >/dev/null 2>&1; then
            echo "Tag $VERSION already exists. Skipping."
            exit 0
          fi

          git tag "$VERSION"
          git push origin "$VERSION"

      - name: Upload docker-compose.yml as artifact
        uses: actions/upload-artifact@ea165f8d65b6e75b540449e92b4886f43607fa02 #v4.6.2
        with:
          name: docker-compose
          path: Docker/docker-compose.yml
          retention-days: 14

  deploy:
    needs: [ build-and-publish-image ]
    permissions:
      contents: read
      packages: read
    uses: ./.github/workflows/deploy.yml
    with:
      environment: "test"
      version: ${{ needs.build-and-publish-image.outputs.version }}
    secrets:
      migration_database_username: ${{ secrets.MIGRATION_DATABASE_USERNAME }}
      migration_database_password: ${{ secrets.MIGRATION_DATABASE_PASSWORD }}
      orchestrator_database_username: ${{ secrets.ORCHESTRATOR_DATABASE_USERNAME }}
      orchestrator_database_password: ${{ secrets.ORCHESTRATOR_DATABASE_PASSWORD }}
      ssh_key: ${{ secrets.SSH_PRIVATE_KEY }}
```

Abbildung 21: Versionierung und Deployaufruf

```yaml
- name: Prepare deployment directory
  shell: bash
  env:
    DEPLOYMENT_FOLDER_NAME: ${{ vars.DEPLOYMENT_FOLDER_NAME }}
  run: |
    set -Eeuo pipefail
    IFS=$'\n\t'
    set -f

    fail() {
      echo "ERROR: $1" >&2
      exit 1
    }

    [[ -n "$DEPLOYMENT_FOLDER_NAME" ]] || fail "DEPLOYMENT_FOLDER_NAME must not be empty."

    [[ -f "Docker/docker-compose.yml" ]] || fail "Docker/docker-compose.yml not found."
    [[ -f ".env.Common" ]] || fail ".env.Common not found."
    [[ -f ".env.Orchestrator" ]] || fail ".env.Orchestrator not found."
    [[ -f ".env.Migrations" ]] || fail ".env.Migrations not found."

    rm -rf -- "$DEPLOYMENT_FOLDER_NAME"
    mkdir -p -- "$DEPLOYMENT_FOLDER_NAME"

    cp -- "Docker/docker-compose.yml" "$DEPLOYMENT_FOLDER_NAME/docker-compose.yml"
    cp -- ".env.Common" "$DEPLOYMENT_FOLDER_NAME/.env.Common"
    cp -- ".env.Orchestrator" "$DEPLOYMENT_FOLDER_NAME/.env.Orchestrator"
    cp -- ".env.Migrations" "$DEPLOYMENT_FOLDER_NAME/.env.Migrations"

- name: Deploy via SSH
  uses: ./.github/actions/ssh-deploy
  with:
    ssh_key: ${{ secrets.SSH_KEY }}
    ssh_user: ${{ vars.SSH_USER }}
    host: ${{ vars.HOST_DEPLOYMENT }}
    docker_registry: ${{ vars.DOCKER_REGISTRY }}
    registry_user: ${{ github.actor }}
    registry_password: ${{ secrets.GITHUB_TOKEN }}
    deploy_dir: /docker/${{ vars.SSH_USER }}/deploy
    deployment_folder_name: ${{ vars.DEPLOYMENT_FOLDER_NAME }}
    source_dir: ${{ vars.DEPLOYMENT_FOLDER_NAME }}
```

Abbildung 22: Deployment mit SSH

A.10 Entwicklerdokumentation

nightcrawler

Customer contact data synchronization between user-service and HubSpot

- Guideline & Conventions
- Architecture

- .NET 10.0 SDK
- Docker
- PostgreSQL database
  - Local installation
  - Docker image: `postgres:16.0-bookworm`

Quick Start IDE

1. Ensure prerequisites are installed
2. Clone the repository
3. Configure the application using environment variables or appsettings
4. Run migrations: `dotnet run --project Infrastructure.PostgreSql.Migrations`
5. Start the service: `dotnet run --project Orchestrator`

Project Structure

```
nightcrawler/
├── Application/                          # Application services & Business logic
├── Domain/                               # Domain models and interfaces
├── Infrastructure.HubSpot/               # HubSpot implementation
├── Infrastructure.PostgreSql/            # PostgreSQL implementation
├── Infrastructure.PostgreSql.Migrations/ # PostgreSQL migrations (console app)
├── Infrastructure.RabbitMQ/              # RabbitMQ implementation
├── Orchestrator/                         # Application composition and startup
├── Presentation.RabbitMQ/                # RabbitMQ consumer
├── *.Tests/                              # Unit Test projects
└── *.IntegrationTests/                   # Integration Test projects
```

Abbildung 23: Ausschnitt Entwicklerdokumentation